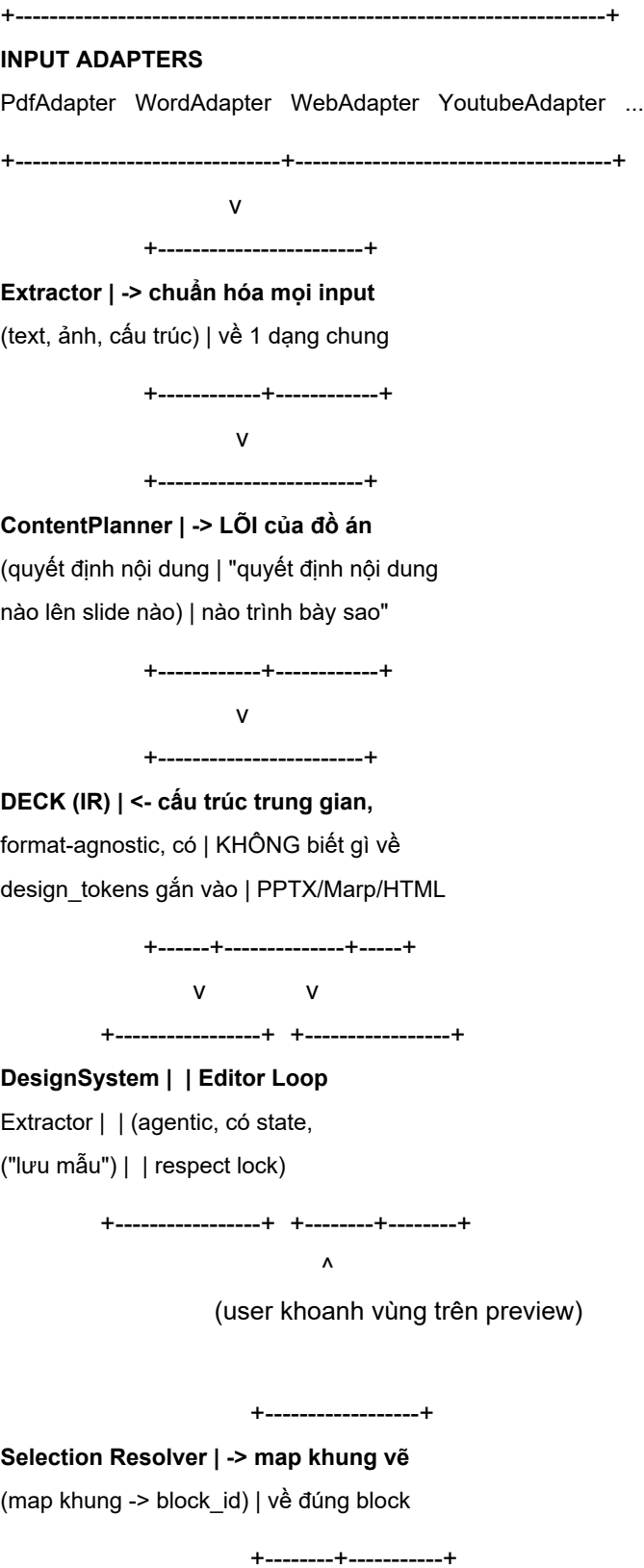
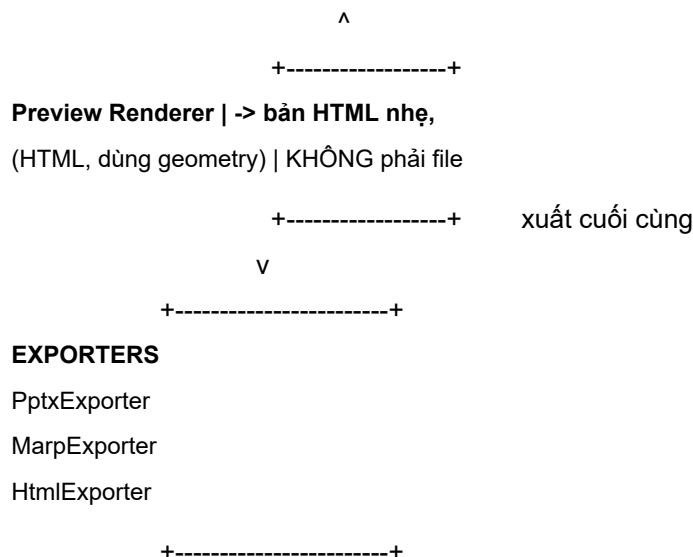


Kiến trúc tổng thể - AI Slide Deck Agent

File này gom lại toàn bộ quyết định kiến trúc đã thảo luận, kèm note giải thích "tại sao" ở mỗi phần. Đọc theo thứ tự từ trên xuống.

1. Sơ đồ tổng quan





Note: Preview Renderer và Selection Resolver là 2 module thuộc giai đoạn mở rộng (sau khi sườn chạy ổn), không nằm trong vertical slice đầu tiên - xem mục 5 (roadmap) và mục 7 (tính năng khoanh vùng) để biết khi nào nên code phần này.

Note: Điểm mấu chốt của toàn bộ sơ đồ là Deck (IR) nằm giữa. Mọi thứ bên trái Deck (input, extract, plan nội dung) không được biết output cuối là gì. Mọi thứ bên phải Deck (export) không được biết input gốc là gì. Tách như vậy thì thêm input mới hay output mới đều không đụng vào phần lõi.

2. Các module chính - mô tả + lý do tách riêng

2.1 Input Adapters

Mỗi loại input (PDF, Word, web link, YouTube) là 1 adapter riêng, nhiệm vụ duy nhất: đọc nguồn và trả về text thô + ảnh thô + metadata (không xử lý nội dung gì cả).

Note: Tách riêng vì mỗi loại input có cách đọc hoàn toàn khác nhau (PDF cần OCR/parse layout, YouTube cần lấy transcript + timestamp). Nhưng output của adapter phải cùng 1 schema để bước sau xử lý thống nhất.

2.2 Extractor

Chuẩn hóa dữ liệu thô từ adapter thành 1 cấu trúc nội dung có phân cấp (heading, đoạn văn, ảnh kèm caption, bảng...). Đây là bước "hiểu" tài liệu, chưa phải "quyết định slide".

2.3 ContentPlanner - phần lõi, khó nhất

Nhận nội dung đã chuẩn hóa, quyết định:

- Chia thành bao nhiêu slide
- Slide nào nói gì (dựa trên loại slide: giảng dạy / catch-up / speaker-support)
- Layout logic nào phù hợp (không phải layout PPTX cụ thể, mà là abstraction như "title+bullets", "image+caption")

Output ra Deck.

Note: Đây là phần bạn nên tập trung nghiên cứu nhiều nhất (matching với hướng "design-token consistency" bạn

đang làm), vì đây là chỗ AI thật sự "quyết định" thay vì chỉ format lại text.

2.4 DesignSystemExtractor

Đọc 1 slide deck mẫu cũ (user upload) hoặc theme có sẵn, trích ra design_tokens (màu, font, spacing, style component). Chạy độc lập, kết quả tái sử dụng được nhiều lần.

Note: Đây chính là cơ chế "lưu mẫu" thầy yêu cầu, và cũng là câu trả lời cho "Lock design?" - design token được extract 1 lần, áp dụng lại cho mọi lần generate sau, thay vì AI tự bịa design mỗi lần.

2.5 Deck (Intermediate Representation)

Cấu trúc dữ liệu trung gian - xem chi tiết ở mục 3.

2.6 Editor Loop

Vòng lặp agent xử lý chỉnh sửa sau khi Deck đã tạo: nhận lệnh sửa (từ chat hoặc click trực tiếp), áp dụng lên đúng phần tử, tôn trọng các phần user đã "annotate lock" (không cho agent đụng vào).

Note: Nên có session state lưu lại Deck hiện tại + lịch sử edit, không phải generate lại từ đầu mỗi lần sửa.

2.7 Exporters

Mỗi định dạng output 1 module riêng, đọc Deck -> sinh file. Thêm định dạng mới = viết thêm 1 exporter, không sửa gì ở trên.

2.8 Preview Renderer (giai đoạn mở rộng)

Render Deck thành 1 bản HTML nhẹ để hiển thị trong UI - không phải file xuất cuối, chỉ để user xem và tương tác (khoanh vùng, click chọn). Dùng chung field geometry (mục 3) với Exporter thật để đảm bảo vị trí khớp nhau tuyệt đối.

Note: Tách khỏi Exporter vì mục đích khác nhau: Exporter tối ưu cho file đúng chuẩn (PPTX/Marp/HTML tải về), Preview tối ưu cho tương tác nhanh trong trình duyệt.

2.9 Selection Resolver (giai đoạn mở rộng)

Khi user vẽ 1 khung chọn trên Preview, module này map khung đó (tọa độ) về đúng block_id trong Deck, dựa trên overlap giữa khung vẽ và geometry của từng block (ngưỡng đề xuất: $\text{overlap} \geq 50\%$ diện tích block thì tính là được chọn). Kết quả trả về danh sách block_id cho Editor Loop xử lý tiếp.

Note: Đây là cách hiện thực hóa yêu cầu "khoanh vùng để sửa" - xem thêm mục 7.

3. Schema Deck (IR) - bản nháp

```
Deck = {  
  "meta": {  
    "title": "...",
```

```

"language": "vi" | "en",
"slide_type": "teaching" | "catchup" | "speaker_support",
"time_limit_minutes": 15 | None,
},
"design_tokens": {
  "colors": {"primary": "#...", "background": "#...", ...},
  "font": {"heading": "...", "body": "..."},
},
"slides": [
  {
    "id": "slide_1",
    "layout": "title+bullets",    # abstraction, không phải PPTX layout id
    "title": "...",
    "blocks": [
      {
        "type": "text", "content": "...", "role": "bullet",
        "geometry": {"x": 0.1, "y": 0.2, "width": 0.4, "height": 0.15}, # % theo slide, thêm ở giai đoạn mở rộng
      },
      {"type": "image", "source_ref": "...", "caption": "..."},
    ],
    "speaker_notes": "...",
    "locked_by_user": False,    # cho annotate-lock
    "source_ref": "page_3_paragraph_2", # trace về nguồn gốc, phục vụ "dùng nguồn chính đã cung cấp"
  }
]
}

```

Note: Field `source_ref` quan trọng vì thầy yêu cầu "sử dụng nguồn chính đã cung cấp" - cần trace được mỗi block nội dung đến từ đâu trong tài liệu gốc, phục vụ cả việc audit lẫn việc user click vào để xem lại nguồn.

>

Note: Field `geometry` không cần có ở vertical slice đầu tiên - chỉ thêm khi bắt đầu code Preview Renderer + Selection Resolver (giai đoạn mở rộng, mục 7). Thêm field mới vào Deck phải qua người giữ schema duyệt (xem file `phan-cong-cong-viec.md`).

4. Tech stack đề xuất theo layer

Layer | Đề xuất | Lý do

Input Adapters + Extractor | Python | thư viện xử lý PDF/Word/scraping mạnh (pypdf, python-docx, BeautifulSoup)

ContentPlanner | Python, gọi thẳng LLM API (không dùng framework nặng như LangChain) | dễ debug, dễ giải trình với thầy cô về luồng chạy thật

Deck (IR) | JSON schema thuần, validate bằng Pydantic | rõ ràng, dễ test độc lập từng module

Renderer/Exporter | python-pptx (đã chốt), sau thêm Marp (markdown thuần), HTML (Jinja2 template) | cùng ngôn ngữ Python xuyên suốt backend

Editor Loop / session state | Python backend + WebSocket hoặc REST polling | cần state persistent giữa các lần sửa
Frontend | tùy chọn sau (React phổ biến nhất) | không ảnh hưởng core, đổi được bất cứ lúc nào

Note: Giữ 1 ngôn ngữ (Python) xuyên suốt backend để giảm chi phí giao tiếp giữa các module trong team 4 người.

5. Thứ tự triển khai đề xuất (roadmap)

1. Định nghĩa schema Deck - cả team thống nhất, viết bằng Pydantic để validate.
2. Vertical slice tối thiểu: 1 input (PDF) -> Extractor -> ContentPlanner (rule đơn giản trước, chưa cần AI phức tạp) -> Deck (in ra JSON kiểm tra) -> PptxExporter -> file PPTX thật.
3. Demo sớm cho thầy - dù thô, để lấy feedback UX.
4. Thêm DesignSystemExtractor (đọc 1 mẫu cũ -> design_tokens) -> áp vào PptxExporter.
5. Thêm Editor Loop (annotate-lock, chỉnh sửa qua chat).
6. Thêm exporter thứ 2 (Marp hoặc HTML) - bước validate quan trọng, kiểm tra Deck schema có đủ tổng quát không.
7. Mở rộng input adapters (Word, web, YouTube).
8. Thêm geometry vào Deck + Preview Renderer + Selection Resolver (tính năng khoanh vùng để sửa - xem mục 7 bên dưới).
9. Các phần còn lại: Anh-Việt, giới hạn thời gian present, AI sinh ảnh, hỗ trợ trình chiếu.

Thứ tự ưu tiên thực tế từ bước 4 trở đi nên dựa trên feedback thật của thầy sau demo, không cố định cứng theo danh sách này - xem file `phan-cong-cong-viec.md` để biết cách chia việc theo 2 giai đoạn.

6. Quyết định "đắt" (chốt sớm) vs "rẻ" (để sau)

Chốt trước khi code (để sau = refactor sâu):

- Schema Deck (IR)
- Module boundaries (Planner tách khỏi Renderer tách khỏi Exporter)
- Ngôn ngữ backend chính (Python)
- Cách gọi LLM (tự viết orchestration, không dùng framework nặng)

Để sau, không ảnh hưởng kiến trúc lõi:

- Theme cụ thể
 - Frontend framework
 - Database cụ thể
 - Deploy/hosting
 - Thứ tự thêm input/output types khác (miễn interface đã chuẩn)
-

7. Tính năng khoanh vùng để sửa (chi tiết)

Luồng xử lý: User vẽ khung trên Preview -> Selection Resolver map khung -> block_id -> Editor Loop áp dụng edit vào Deck (tôn trọng locked_by_user) -> Deck cập nhật -> render lại Preview + Export.

2 điểm rủi ro cần lưu ý khi code:

1. Ambiguity khi khoanh trúng nhiều/1 phần block - dùng ngưỡng overlap cố định ($\geq 50\%$ diện tích), tránh xử lý "cắt 1 phần block" ở giai đoạn đầu.
 2. Preview và Export phải khớp tuyệt đối vị trí - cả 2 đều phải đọc từ cùng field geometry, không tự tính layout riêng, nếu không user khoanh đúng chỗ trên Preview nhưng Editor sửa sai chỗ trên file thật.
-

8. Evaluation (đánh giá hệ thống)

Phương pháp: kết hợp MLLM-as-judge (theo hướng PresentBench - checklist chi tiết riêng cho từng input, judge chấm từng mục kèm bằng chứng) với human annotation trên mẫu con để kiểm tra độ tin cậy (đo tương quan điểm judge vs người).

Các khía cạnh chấm:

- Content fidelity (đúng/đủ nội dung so với nguồn, không bịa)
- Ready-to-use (không lỗi layout, chữ tràn, hình vỡ - khớp yêu cầu "1 prompt dùng được luôn")
- Design consistency (tuân theo design token đã lock)
- Structure (số slide, cách chia nội dung hợp lý với loại slide)

Lưu ý khi báo cáo: nêu rõ hạn chế self-preference bias của MLLM-judge (nên dùng model khác họ để generate và để judge), không chỉ báo cáo con số mà không nhắc giới hạn.

Chi tiết phân công phần này: xem file phan-cong-cong-viec.md (người D).